

Top-Down Market Report

Analysis window: 2025-01-01 to present | **Data source:** Yahoo Finance via `yfinance`

How to read this report

The analysis moves from broad to narrow, in three steps:

Step	Question	Section
1. Market	Which sectors led and which lagged?	1. Sector Landscape
2. Sector	Is that leadership durable or a narrow spike?	2. Sector Drilldown
3. Security	Does one name inside the leader look attractive?	3. Micron (MU)

Each section presents a chart, a table of the numbers behind it, and a short written conclusion. **Conclusions are generated from the data at run time**, not typed in by hand, so they stay correct when the notebook is re-run at a later date.

How to run

Execute top to bottom. Every cell depends only on cells above it. Runtime is roughly one to two minutes, almost all of it network calls to Yahoo.

Methodology and definitions

Universe. One ETF per sector, taken as the largest fund Yahoo lists for that sector. Technology is overridden to XLK: Yahoo ranks QQQ first by assets, but QQQ tracks the Nasdaq-100, not the technology sector, and its holdings overlap consumer-cyclical (AMZN, TSLA) and communication-services (GOOGL, META). Using it would double-count those sectors.

Prices. Daily closes with `auto_adjust=True`, so every series is adjusted for splits and dividends. Returns here are total returns, not price returns.

Lookback vs. display window. Prices are downloaded from 2024-03-01 but charted from 2025-01-01. The extra history exists so the 200-day moving average is fully populated on the first displayed bar. Without it the SMA-200 would be blank for roughly the first ten months of the chart.

Standardisation. Sector series shown on a shared axis are rebased so each starts at 100 on the first display date. A value of 118 means a position opened on 2025-01-01 is worth 18% more. Raw prices are never plotted together: a fund near 40 and one near 260 on the same axis compresses the smaller series into a flat line and makes the comparison meaningless.

Indicator definitions.

Indicator	Definition
SMA-50 / SMA-200	Simple mean of the trailing 50 / 200 closes. Trend reference.
EMA-8 / EMA-21	Exponentially weighted mean, spans 8 and 21. Faster trend reference.
RSI-14	Wilder's Relative Strength Index: average gain over average loss across 14 days, smoothed at alpha = 1/14, scaled 0-100.
Annualised volatility	Standard deviation of daily returns times the square root of 252.
Maximum drawdown	Largest peak-to-trough decline inside the display window.

Reading RSI. Above 70 is conventionally called overbought and below 30 oversold, but in a sustained uptrend RSI can hold above 70 for weeks. Treat it as a momentum gauge, not a signal to act on in isolation.

Limitations. Yahoo is a free, unwarranted feed; fundamentals occasionally carry restatement errors and analyst fields are sparse. This report is descriptive: no forecast, no valuation model, no recommendation.

0. Setup

```
In [1]: import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from IPython.display import Markdown, display

# ---- Configuration -----
DISPLAY_START = "2025-01-01" # first date shown on charts
LOOKBACK_START = "2024-03-01" # earlier fetch so SMA-200 is warm at DISPLAY_START
FOCUS_TICKER = "MU"
TRADING_DAYS = 252
```

```

# ---- Chart style -----
plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.dpi": 130,
    "font.size": 11,
    "axes.titlesize": 13,
    "axes.titleweight": "bold",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "legend.frameon": False,
})

ACCENT = "#4C72B0"
POSITIVE = "#55A868"
NEGATIVE = "#C44E52"
MUTED = "#BAB6B0"

def pct(x, dp=1):
    return f"{x:,.{dp}f}%"

def money(x):
    for cut, suf in ((1e12, "T"), (1e9, "B"), (1e6, "M")):
        if abs(x) >= cut:
            return f"${x/cut:,.2f}{suf}"
    return f"${x:,.0f}"

def note(text):
    # Render a runtime-generated conclusion as real markdown, so it
    # survives nbconvert --no-input as prose rather than as stdout.
    display(Markdown(text))

print("Setup complete.")

```

Setup complete.

Safe fetch wrapper

`yfinance` raises on individual endpoints fairly often, and a property that works today can 404 tomorrow. Because this notebook is exported non-interactively with `nbconvert --execute`, a single uncaught exception would abort the whole render. Every optional pull below goes through `safe`, which degrades to `None` and prints a warning instead of stopping the run.

```
In [2]: def safe(fn, label, default=None):
        # Call fn(); on failure warn and return default rather than raising.
        try:
            out = fn()
            if out is None or (hasattr(out, "empty") and out.empty):
                print(f"[warn] {label}: returned empty")
                return default
            return out
        except Exception as exc:
            print(f"[warn] {label}: {type(exc).__name__} - {exc}")
            return default
```

Analytics helpers

`rsi` uses Wilder smoothing via `ewm(alpha=1/period)`, which matches the convention on charting platforms. A plain `rolling(14).mean()` produces a noticeably choppier series and is a common source of mismatch when cross-checking against a broker's chart.

```
In [3]: def rsi(close, period=14):
        # Wilder's RSI.
        delta = close.diff()
        gain = delta.clip(lower=0)
        loss = -delta.clip(upper=0)
        avg_gain = gain.ewm(alpha=1/period, adjust=False, min_periods=period).mean()
        avg_loss = loss.ewm(alpha=1/period, adjust=False, min_periods=period).mean()
        return 100 - 100 / (1 + avg_gain / avg_loss)

def rebase(px_, base=100.0):
    # Index every column to `base` at the first row.
    return px_ / px_.iloc[0] * base

def performance_table(px_):
    # Return / risk summary over the supplied window.
    rets = px_.pct_change()
    reb_ = rebase(px_)
    out = pd.DataFrame({
        "total_return_pct": reb_.iloc[-1] - 100,
        "ann_vol_pct": rets.std() * np.sqrt(TRADING_DAYS) * 100,
        "max_drawdown_pct": (px_ / px_.cummax() - 1).min() * 100,
    })
    # Column names deliberately avoid '%': a '%' in a name blocks attribute
    # access (df.total_return_% is a SyntaxError) and forces bracket indexing.
    out["return_per_unit_risk"] = out["total_return_pct"] / out["ann_vol_pct"]
    return out.round(2).sort_values("total_return_pct", ascending=False)

def get_close(tickers, start, **kw):
    # Adjusted closes as a tidy DataFrame regardless of ticker count.
    raw = yf.download(tickers, start=start, auto_adjust=True, progress=False, **kw)
    close = raw["Close"]
    if isinstance(close, pd.Series):
```

```
name = tickers if isinstance(tickers, str) else tickers[0]
close = close.to_frame(name)
return close.dropna(how="all")
```

1. Sector Landscape

Question: across the eleven sectors, where did capital actually compound over the window, and how much volatility was endured to get there?

1.1 Building the universe

Sector keys come from `yfinance`'s own constant rather than a hardcoded list. Yahoo uses `financial-services` and `basic-materials` — not `financials` and `materials` — and sourcing the list from the package removes the guesswork.

```
In [4]: from yfinance.const import SECTOR_INDUSTY_MAPPING_LC  # the typo is in the library

sector_keys = list(SECTOR_INDUSTY_MAPPING_LC.keys())

top_etfs = {}
for key in sector_keys:
    res = safe(lambda k=key: yf.Sector(k).top_etfs, f"sector {key}")
    if res:
        top_etfs[key] = res

# Largest fund per sector, with the Technology override documented above.
best = {k: next(iter(v)) for k, v in top_etfs.items()}
best["technology"] = "XLK"

universe = pd.Series(best, name="etf").rename_axis("sector").to_frame()
universe["etf_name"] = [top_etfs[s].get(t, "") for s, t in best.items()]
universe
```

Out[4]:

	etf	etf_name
sector		
basic-materials	XLB	State Street Materials Select S
communication-services	XLC	State Street Communication Serv
consumer-cyclical	XLY	State Street Consumer Discretio
consumer-defensive	XLP	State Street Consumer Staples S
energy	XLE	State Street Energy Select Sect
financial-services	XLF	State Street Financial Select S
healthcare	XLV	State Street Health Care Select
industrials	XLI	State Street Industrial Select
real-estate	VNQ	Vanguard Real Estate ETF
technology	XLK	State Street Technology Select
utilities	XLU	State Street Utilities Select S

Caveat on deeper slices. Yahoo's sector tagging is loose. The `financial-services` list carries NVDL — a 2x leveraged single-stock NVDA product — at rank five, and other sectors surface SOXL, FAS, NAIL and UYM. Taking only the first entry per sector avoids all of them, but widening to a top-N would need an explicit leverage filter.

1.2 Price history

```
In [5]: px_full = get_close(sorted(best.values()), LOOKBACK_START)
px = px_full.loc[DISPLAY_START:]

sym_to_sector = {v: k for k, v in best.items()}
px = px[[s for s in best.values() if s in px.columns]]

print(f"{px.shape[1]} sectors | {px.index.min().date()} to {px.index.max().date()}
      f"| {len(px)} trading days")
px.tail(3).round(2)
```

11 sectors | 2025-01-02 to 2026-08-04 | 397 trading days

Out[5]:

Ticker	XLB	XLC	XLY	XLP	XLE	XLF	XLV	XLI	VNQ	XLK	XLU
Date											
2026-07-31	50.43	108.24	116.09	85.05	59.55	56.94	162.55	179.84	98.95	175.35	44.35
2026-08-03	51.01	111.34	118.21	84.86	58.79	57.38	162.24	183.16	99.07	178.04	44.36
2026-08-04	52.00	112.04	118.29	85.37	58.52	57.88	162.10	186.40	98.92	186.90	44.11

1.3 Relative performance

All series are rebased to 100 at the first display date. Leader and laggard are highlighted and everything between them is greyed, so the spread — which is the actual story — is legible at a glance. Series are labelled directly at the right edge because an eleven-entry legend is slower to read than a label on the line.

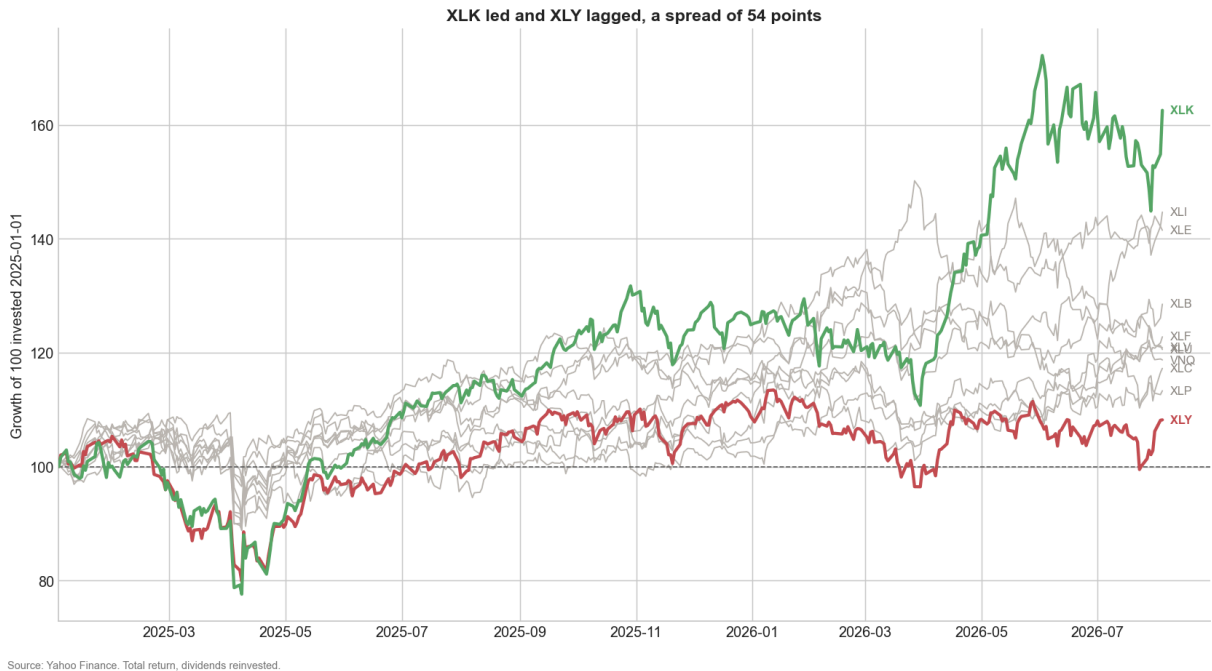
```
In [6]: reb = rebase(px)
perf = performance_table(px)

leader, laggard = perf.index[0], perf.index[-1]
spread = perf["total_return_pct"].iloc[0] - perf["total_return_pct"].iloc[-1]

fig, ax = plt.subplots(figsize=(13, 7))
for sym in reb.columns:
    if sym == leader:
        ax.plot(reb.index, reb[sym], color=POSITIVE, lw=2.4, zorder=3)
    elif sym == laggard:
        ax.plot(reb.index, reb[sym], color=NEGATIVE, lw=2.4, zorder=3)
    else:
        ax.plot(reb.index, reb[sym], color=MUTED, lw=1.0, zorder=1)

for sym in reb.columns:
    colour = POSITIVE if sym == leader else NEGATIVE if sym == laggard else "#8A858"
    weight = "bold" if sym in (leader, laggard) else "normal"
    ax.annotate(sym, (reb.index[-1], reb[sym].iloc[-1]),
                xytext=(6, 0), textcoords="offset points",
                va="center", fontsize=9, color=colour, fontweight=weight)

ax.axhline(100, color="#555", ls="--", lw=0.9)
ax.set_title(f"{leader} led and {laggard} lagged, a spread of {spread:,.0f} points")
ax.set_ylabel(f"Growth of 100 invested {DISPLAY_START}")
ax.set_xlabel("")
ax.set_xlim(reb.index[0], reb.index[-1] + pd.Timedelta(days=25))
fig.text(0.01, -0.02,
        "Source: Yahoo Finance. Total return, dividends reinvested.",
        fontsize=8, color="#777")
plt.tight_layout()
plt.show()
```



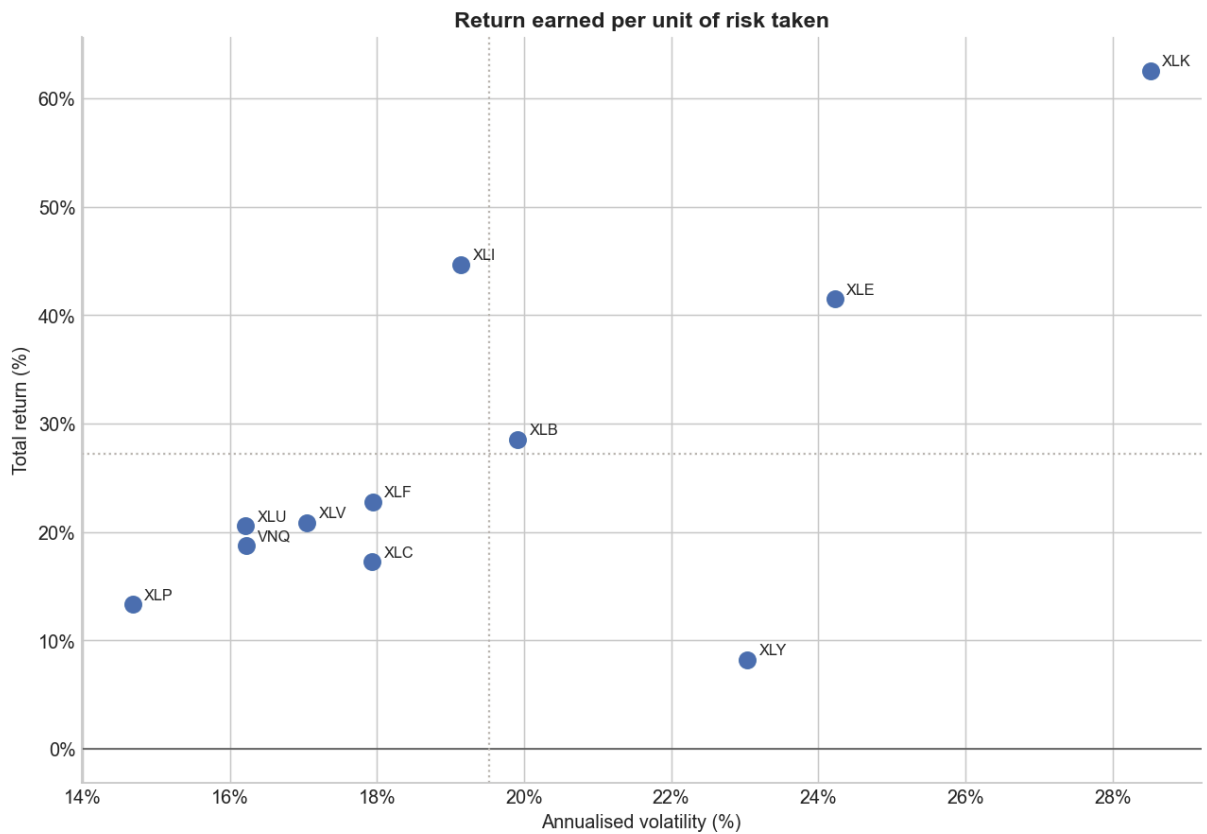
1.4 Return against risk

Total return on its own rewards whoever took the most risk. Plotting return against annualised volatility separates real outperformance from exposure to a volatile factor. Points toward the upper-left earned more per unit of risk.

```
In [7]: fig, ax = plt.subplots(figsize=(10, 7))
ax.scatter(perf["ann_vol_pct"], perf["total_return_pct"], s=90, color=ACCENT, zorder=1)
for sym, row in perf.iterrows():
    ax.annotate(sym, (row["ann_vol_pct"], row["total_return_pct"]),
                xytext=(7, 3), textcoords="offset points", fontsize=9)

ax.axhline(0, color="#555", lw=0.9)
ax.axhline(perf["total_return_pct"].mean(), color=MUTED, ls=":", lw=1.2)
ax.axvline(perf["ann_vol_pct"].mean(), color=MUTED, ls=":", lw=1.2)
ax.set_xlabel("Annualised volatility (%)")
ax.set_ylabel("Total return (%)")
ax.set_title("Return earned per unit of risk taken")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v, p: f"{v:,.0f}%"))
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda v, p: f"{v:,.0f}%"))
fig.text(0.01, -0.02, "Dotted lines mark the cross-sector average on each axis.",
        fontsize=8, color="#777")
plt.tight_layout()
plt.show()

perf.assign(sector=[sym_to_sector.get(s, "") for s in perf.index])[
    ["sector", "total_return_pct", "ann_vol_pct", "max_drawdown_pct", "return_per_u
]
```

Dotted lines mark the cross-sector average on each axis.

Out[7]:

sector	total_return_pct	ann_vol_pct	max_drawdown_pct	return_per_unit_risk
--------	------------------	-------------	------------------	----------------------

Ticker				
XLK	technology	62.54	28.51	-25.66
XLI	industrials	44.68	19.14	-17.57
XLE	energy	41.49	24.22	-18.79
XLB	basic-materials	28.52	19.91	-17.99
XLF	financial-services	22.78	17.94	-15.54
XLV	healthcare	20.89	17.05	-13.64
XLU	utilities	20.57	16.21	-9.18
VNQ	real-estate	18.73	16.22	-14.55
XLC	communication-services	17.25	17.93	-17.97
XLP	consumer-defensive	13.32	14.68	-9.69
XLY	consumer-cyclical	8.21	23.03	-24.17

1.5 Conclusion

```
In [8]: above = int((perf["total_return_pct"] > 0).sum())
eff      = perf.sort_values("return_per_unit_risk", ascending=False)
breadth  = "broad-based" if above >= 8 else "narrow" if above <= 4 else "mixed"
worst_dd = perf["max_drawdown_pct"].idxmin()

note(
f"""
**Sector landscape — findings**

- **{leader}** ({sym_to_sector.get(leader, '')}) led with
**{pct(perf.loc[leader, 'total_return_pct'])}**, while
**{laggard}** ({sym_to_sector.get(laggard, '')}) trailed at
**{pct(perf.loc[laggard, 'total_return_pct'])}**. The **{spread:,.0f}-point spread**
is the dispersion an allocator was paid for getting right.
- Participation was **{breadth}**: **{above} of {len(perf)}** sectors finished
above their starting value.
- On a risk-adjusted basis the ranking shifts. **{eff.index[0]}** delivered the
most return per unit of volatility ({eff['return_per_unit_risk'].iloc[0]:.2f});
**{eff.index[-1]}** the least ({eff['return_per_unit_risk'].iloc[-1]:.2f}).
- The deepest peak-to-trough decline belonged to **{worst_dd}** at
**{pct(perf['max_drawdown_pct'].min())}** — the drawdown a holder had to sit
through, which never appears in a headline return figure.

**So what:** dispersion of this size means sector selection, not market
direction, drove returns over the window. That is what justifies drilling into
the leader rather than simply buying the index.
"""
)
```

Sector landscape — findings

- **XLK** (technology) led with **62.5%**, while **XLY** (consumer-cyclical) trailed at **8.2%**. The **54-point spread** is the dispersion an allocator was paid for getting right.
- Participation was **broad-based: 11 of 11** sectors finished above their starting value.
- On a risk-adjusted basis the ranking shifts. **XLI** delivered the most return per unit of volatility (2.33); **XLY** the least (0.36).
- The deepest peak-to-trough decline belonged to **XLK** at **-25.7%** — the drawdown a holder had to sit through, which never appears in a headline return figure.

So what: dispersion of this size means sector selection, not market direction, drove returns over the window. That is what justifies drilling into the leader rather than simply buying the index.

2. Sector Drilldown — Technology

Question: technology led. Is that leadership steady, or the residue of a short violent move that has already faded?

A trailing return is one number and hides its own path — two sectors can finish in the same place having behaved completely differently. The rolling relative-strength line below shows *when* the lead was built.

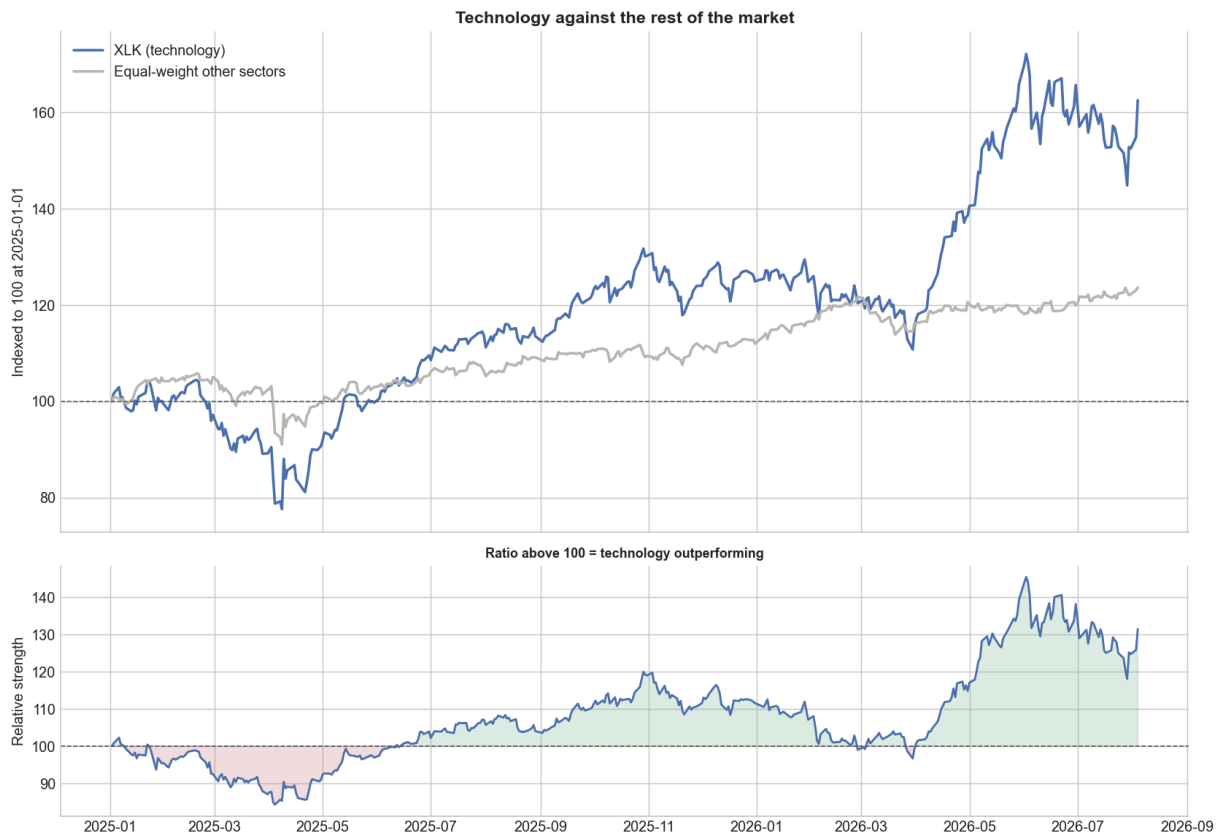
```
In [9]: TECH = best["technology"]
peers = [s for s in px.columns if s != TECH]

# Equal-weight basket of every other sector = "the rest of the market".
rest = rebase(px[peers]).mean(axis=1)
tech_reb = rebase(px[[TECH]])[TECH]
rs = tech_reb / rest * 100

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(13, 9), sharex=True,
                               gridspec_kw={"height_ratios": [2, 1]})

ax1.plot(tech_reb.index, tech_reb, color=ACCENT, lw=2, label=f"{TECH} (technology)")
ax1.plot(rest.index, rest, color=MUTED, lw=2, label="Equal-weight other sectors")
ax1.axhline(100, color="#555", ls="--", lw=0.9)
ax1.set_ylabel(f"Indexed to 100 at {DISPLAY_START}")
ax1.set_title("Technology against the rest of the market")
ax1.legend(loc="upper left")

ax2.plot(rs.index, rs, color=ACCENT, lw=1.6)
ax2.axhline(100, color="#555", ls="--", lw=0.9)
ax2.fill_between(rs.index, 100, rs, where=rs >= 100, color=POSITIVE, alpha=0.18)
ax2.fill_between(rs.index, 100, rs, where=rs < 100, color=NEGATIVE, alpha=0.18)
ax2.set_ylabel("Relative strength")
ax2.set_xlabel("")
ax2.set_title("Ratio above 100 = technology outperforming", fontsize=10)
plt.tight_layout()
plt.show()
```

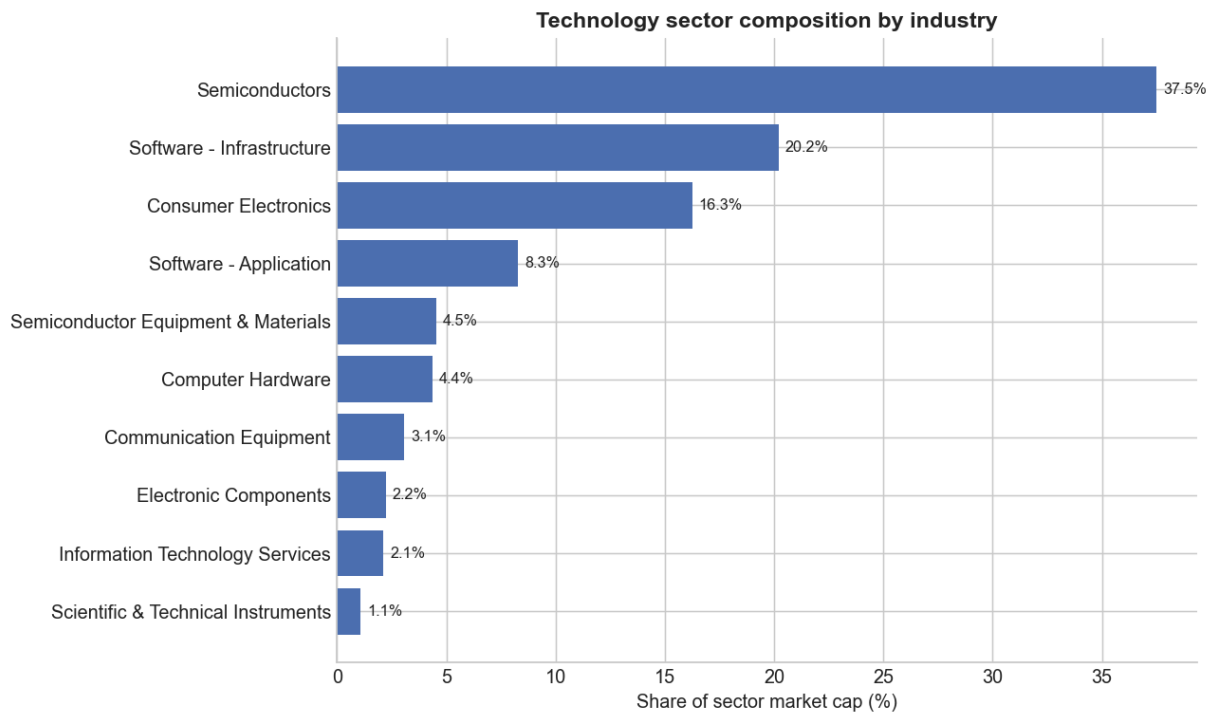


2.1 What is inside the sector

Sector leadership is often concentrated in a single industry. Yahoo exposes industry composition and market weights directly, which shows whether the move was broad or driven by one group.

```
In [10]: tech_sector = safe(lambda: yf.Sector("technology"), "yf.Sector(technology)")
industries = safe(lambda: tech_sector.industries, "technology industries") if tech

if industries is not None:
    wcol = "market weight" if "market weight" in industries.columns else industries
    ind_mix = industries.sort_values(wcol, ascending=False).head(10)
    fig, ax = plt.subplots(figsize=(10, 6))
    bars = ax.barh(ind_mix["name"][::-1], ind_mix[wcol][::-1] * 100, color=ACCENT)
    for b in bars:
        ax.text(b.get_width() + 0.3, b.get_y() + b.get_height() / 2,
                f"{b.get_width():.1f}%", va="center", fontsize=9)
    ax.set_xlabel("Share of sector market cap (%)")
    ax.set_title("Technology sector composition by industry")
    plt.tight_layout()
    plt.show()
    display(ind_mix[["name", wcol]].reset_index(drop=True))
```



	name	market weight
0	Semiconductors	0.374977
1	Software - Infrastructure	0.202003
2	Consumer Electronics	0.162512
3	Software - Application	0.082910
4	Semiconductor Equipment & Materials	0.045274
5	Computer Hardware	0.043755
6	Communication Equipment	0.030686
7	Electronic Components	0.022286
8	Information Technology Services	0.021200
9	Scientific & Technical Instruments	0.010789

2.2 Conclusion

```
In [11]: rs_now          = rs.iloc[-1]
rs_peak_date    = rs.idxmax()
days_since_pk  = (rs.index[-1] - rs_peak_date).days
share_leading   = (rs > 100).mean() * 100
trend           = "still expanding" if rs_now >= rs.rolling(21).mean().iloc[-1] else

verdict = ("appears intact, which supports looking for a security-level "
          "expression of it") if (rs_now > 100 and trend == "still expanding") else
          ("has lost momentum, so any single-name idea inside it needs to stand "
          "on its own merits rather than on a sector tailwind")
```

```

note(
f"""
Technology drilldown – findings

- Relative strength ended at {rs_now:,.1f} against a 100 baseline, meaning
  technology outperformed the equal-weight basket of other sectors by roughly
  {rs_now - 100:,.1f}% over the window.
- The lead held for {share_leading:,.0f}% of trading days, and the ratio is
  currently {trend} relative to its own 21-day average.
- Peak relative strength occurred on {rs_peak_date.date()}, which is
  {days_since_pk} days ago. A recent peak points to live leadership; a distant
  one means this is a move that has already played out.

So what: the leadership {verdict}.
"""
)

```

Technology drilldown — findings

- Relative strength ended at **131.5** against a 100 baseline, meaning technology outperformed the equal-weight basket of other sectors by roughly **31.5%** over the window.
- The lead held for **72%** of trading days, and the ratio is currently **still expanding** relative to its own 21-day average.
- Peak relative strength occurred on **2026-06-02**, which is **63 days** ago. A recent peak points to live leadership; a distant one means this is a move that has already played out.

So what: the leadership appears intact, which supports looking for a security-level expression of it.

3. Security Analysis — Micron Technology (MU)

Fundamentals first, technicals second. The order is deliberate: fundamentals establish whether the business is worth owning, technicals only inform timing. Reversing them invites reading a chart pattern into a deteriorating business.

3.1 Company snapshot

```

In [12]: mu = yf.Ticker(FOCUS_TICKER)
info = safe(lambda: mu.info, "MU info", {}) or {}

def show(key, fmt):
    v = info.get(key)
    try:

```

```

        return fmt(v) if v is not None else "n/a"
    except Exception:
        return "n/a"

rows = [
    ("Name", show("shortName", str)),
    ("Sector", show("sector", str)),
    ("Industry", show("industry", str)),
    ("Market cap", show("marketCap", money)),
    ("Trailing P/E", show("trailingPE", lambda v: f"{v:,.1f}")),
    ("Forward P/E", show("forwardPE", lambda v: f"{v:,.1f}")),
    ("Price / book", show("priceToBook", lambda v: f"{v:,.2f}")),
    ("Profit margin", show("profitMargins", lambda v: pct(v * 100))),
    ("Return on equity", show("returnOnEquity", lambda v: pct(v * 100))),
    ("Revenue growth (yoy)", show("revenueGrowth", lambda v: pct(v * 100))),
    ("Beta", show("beta", lambda v: f"{v:,.2f}")),
    ("52-week range", f"${info.get('fiftyTwoWeekLow', float('nan')):,.2f} - "
                      f"${info.get('fiftyTwoWeekHigh', float('nan')):,.2f}")
]

snapshot = pd.DataFrame(rows, columns=["metric", "value"]).set_index("metric")
snapshot

```

Out[12]:

	value
metric	
Name	Micron Technology, Inc.
Sector	Technology
Industry	Semiconductors
Market cap	n/a
Trailing P/E	18.8
Forward P/E	5.7
Price / book	10.01
Profit margin	55.9%
Return on equity	66.6%
Revenue growth (yoy)	345.7%
Beta	2.21
52-week range	106.75—1,255.00

3.2 Revenue and margin trend

For a memory manufacturer this is the most informative fundamental view available.

Memory is a commodity: pricing swings with the supply cycle, and **gross margin moves far more violently than revenue**. Margin direction identifies where in the cycle the business

currently sits, which a revenue line alone will not show. Unless, we are not in a cycle but an AI revolution?

```
In [13]: qis = safe(lambda: mu.quarterly_income_stmt, "quarterly income statement")

def pick(df, *candidates):
    # Income-statement row labels vary by filer; take the first that exists.
    for c in candidates:
        if c in df.index:
            return df.loc[c]
    return pd.Series(index=df.columns, dtype="float64")

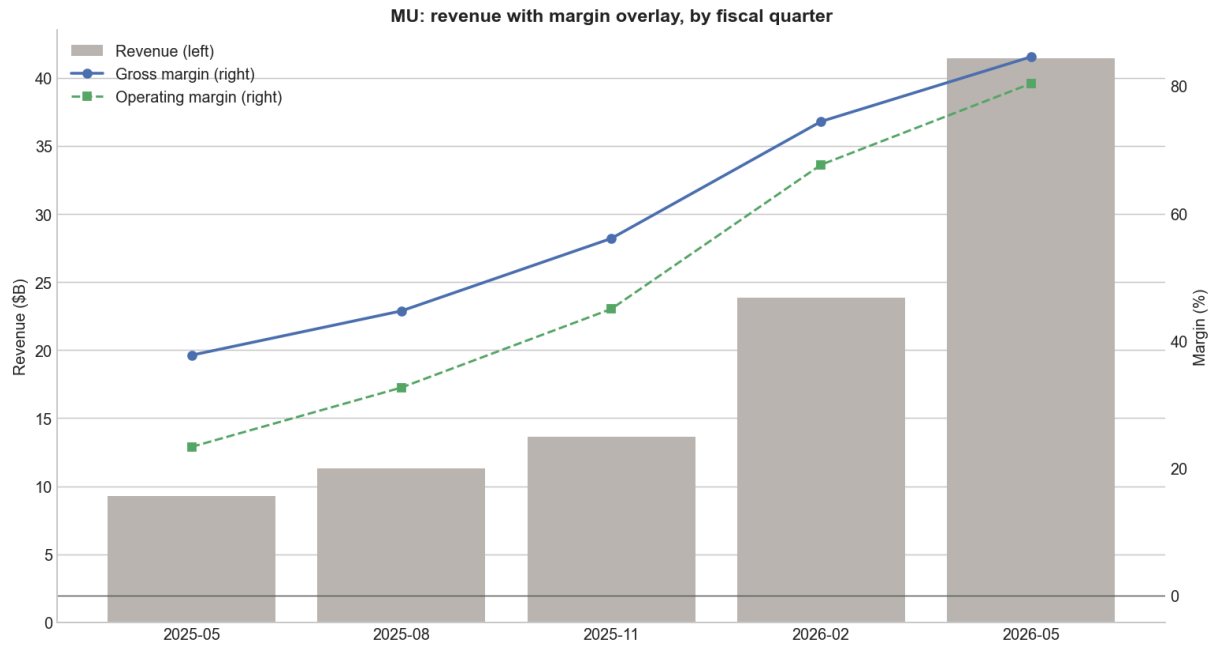
fin = None
if qis is not None:
    fin = pd.DataFrame({
        "revenue": pick(qis, "Total Revenue", "Operating Revenue"),
        "gross_profit": pick(qis, "Gross Profit"),
        "operating_income": pick(qis, "Operating Income", "Total Operating Income A", "Net Income"),
        "net_income": pick(qis, "Net Income", "Net Income Common Stockholders")
    }).sort_index()
    fin["gross_margin_pct"] = fin["gross_profit"] / fin["revenue"] * 100
    fin["operating_margin_pct"] = fin["operating_income"] / fin["revenue"] * 100
    fin["net_margin_pct"] = fin["net_income"] / fin["revenue"] * 100

    fig, axA = plt.subplots(figsize=(12, 6.5))
    labels = [d.strftime("%Y-%m") for d in fin.index]
    axA.bar(labels, fin["revenue"] / 1e9, color=MUTED, label="Revenue (left)")
    axA.set_ylabel("Revenue ($B)")
    axA.grid(False, axis="x")

    axB = axA.twinx()
    axB.plot(labels, fin["gross_margin_pct"], color=ACCENT, marker="o", lw=2,
             label="Gross margin (right)")
    axB.plot(labels, fin["operating_margin_pct"], color=POSITIVE, marker="s", lw=1,
             ls="--", label="Operating margin (right)")
    axB.set_ylabel("Margin (%)")
    axB.grid(False)
    axB.axhline(0, color="#555", lw=0.8)

    hA, lA = axA.get_legend_handles_labels()
    hB, lB = axB.get_legend_handles_labels()
    axA.legend(hA + hB, lA + lB, loc="upper left")
    axA.set_title(f"{FOCUS_TICKER}: revenue with margin overlay, by fiscal quarter")
    fig.text(0.01, -0.03,
            "Dual axis used deliberately: revenue in $B and margin in % share a "
            "time base but not a scale.", fontsize=8, color="#777")
    plt.tight_layout()
    plt.show()

    display(fin[["revenue", "gross_margin_pct", "operating_margin_pct", "net_margin_pct"]
              .assign(revenue=lambda d: (d["revenue"] / 1e9).round(2)).round(1))
```

	revenue	gross_margin_pct	operating_margin_pct	net_margin_pct
2025-05-31	9.3	37.7	23.3	20.3
2025-08-31	11.3	44.7	32.6	28.3
2025-11-30	13.6	56.0	45.0	38.4
2026-02-28	23.9	74.4	67.6	57.8
2026-05-31	41.5	84.6	80.4	68.1

3.3 Forward expectations

Section 3.2 reported what has already happened. These six tables report what the sell side expects next — the bar MU is measured against at the coming report.

Property	What it holds
earnings_estimate	Consensus EPS by period, with analyst count and the low/high band
revenue_estimate	The same for the top line
growth_estimates	MU's expected growth against its industry, sector and the index
eps_trend	How the consensus EPS number has moved over the last 90 days
eps_revisions	How many analysts moved it, up and down, over 7 and 30 days
analyst_price_targets	Low / mean / median / high target against the current price

All are indexed by the same period codes: `0q` current quarter, `+1q` next quarter, `0y` current fiscal year, `+1y` next year.

The interpretive point. The *level* of consensus carries very little information — it is public, and already in the price. The *direction of its revision* carries a great deal. A stock does not rise because consensus EPS is 2.40; *it rises because* 2.40 was \$1.90 ninety days ago and the market has not finished repricing that drift. `eps_trend` and `eps_revisions` are therefore the two tables that matter most here, and they are the ones a naive read of this section skips.

This applies with extra force to Micron. Sell-side models lag memory-cycle turns badly, because the cycle is driven by industry supply decisions that are not visible in a quarterly model. Revisions consequently arrive in long one-directional runs rather than oscillating around a true value — which makes the run direction more informative for MU than it would be for a stable industrial.

Reading the dispersion. Check `numberOfAnalysts` against the `high - low` band before drawing any conclusion from it. A wide band across 30 contributors is genuine disagreement about the cycle. A wide band across 4 is noise.

```
In [14]: est      = safe(lambda: mu.earnings_estimate,      "earnings estimate")
rev_est   = safe(lambda: mu.revenue_estimate,           "revenue estimate")
grow      = safe(lambda: mu.growth_estimates,           "growth estimates")
trend_t   = safe(lambda: mu.eps_trend,                   "eps trend")
revis     = safe(lambda: mu.eps_revisions,               "eps revisions")
targets   = safe(lambda: mu.analyst_price_targets,       "price targets", {}) or {}

for label, tbl in [("EPS consensus by period", est),
                  ("Revenue consensus by period", rev_est),
                  ("Growth: company vs industry vs sector vs index", grow),
                  ("EPS estimate trend (how consensus has moved)", trend_t),
                  ("EPS revisions (how many analysts moved it)", revis)]:
    if tbl is not None:
        print(label)
        display(tbl)

if targets:
    display(pd.Series(targets).to_frame("price_target"))
```

EPS consensus by period

	avg	low	high	yearAgoEps	numberOfAnalysts	growth
period						
0q	31.32898	28.04	37.44	3.03000	31	9.339601
+1q	34.86624	25.64	47.81	4.78000	31	6.294200
0y	73.43364	67.39	80.16	8.29000	36	7.858100
+1y	155.56306	106.89	221.27	73.43364	37	1.118400

Revenue consensus by period

	avg	low	high	numberOfAnalysts	yearAgoRevenue	growth
period						
0q	50838974970	46911000000	59805000000	31	11315000000	3.1
+1q	56711909380	45500000000	75369000000	30	13643000000	3.1
0y	129758113130	122598000000	138764000000	39	37378000000	2.1
+1y	240106235120	101608000000	294160382020	42	129758113130	0.1

Growth: company vs industry vs sector vs index

	stockTrend	indexTrend
period		
0q	9.339601	0.4471
+1q	6.294200	0.2290
0y	7.858100	0.2919
+1y	1.118400	0.1438
LTG	NaN	0.1220

EPS estimate trend (how consensus has moved)

	current	7daysAgo	30daysAgo	60daysAgo	90daysAgo
period					
0q	31.32898	31.28511	24.18138	22.66035	22.26158
+1q	34.86624	34.79689	27.63408	25.15494	24.92469
0y	73.43364	73.44403	73.32474	58.73640	57.96829
+1y	155.56306	153.73953	149.63844	105.11877	100.94313

EPS revisions (how many analysts moved it)

	upLast7days	upLast30days	downLast30days	downLast7Days
period				
0q	1	26	0	0
+1q	24	23	0	0
0y	0	29	1	0
+1y	3	30	0	0

	price_target
current	892.6700
high	2200.0000
low	361.0000
mean	1507.7906
median	1550.0000

3.3.1 Estimate revision momentum

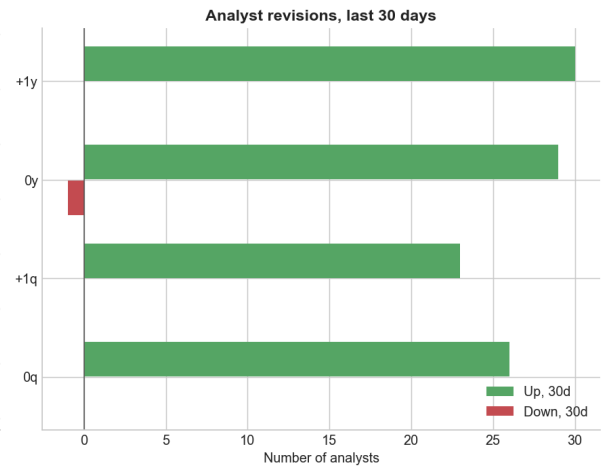
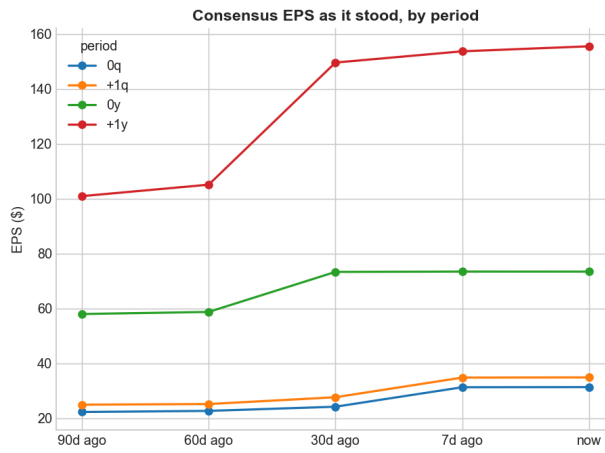
The left panel plots consensus EPS as it stood 90, 60, 30 and 7 days ago against today. An upward-sloping line means analysts have been raising numbers into the print. The right panel shows the breadth behind that move: a rising consensus driven by two analysts is a different thing from one driven by fifteen.

```
In [15]: if trend_t is not None:
    cols = ["90daysAgo", "60daysAgo", "30daysAgo", "7daysAgo", "current"]
    have = [c for c in cols if c in trend_t.columns]
    order = [p for p in ["0q", "+1q", "0y", "+1y"] if p in trend_t.index

    fig, (axL, axR) = plt.subplots(1, 2, figsize=(14, 5.5))
    xlab = [c.replace("daysAgo", "d ago").replace("current", "now") for c in have]

    for p in order:
        series = pd.to_numeric(trend_t.loc[p, have], errors="coerce")
        axL.plot(xlab, series.values, marker="o", lw=1.8, label=p)
    axL.set_title("Consensus EPS as it stood, by period")
    axL.set_ylabel("EPS ($)")
    axL.legend(title="period")

    if revis is not None:
        rp = [p for p in order if p in revis.index]
        ups = pd.to_numeric(revis.loc[rp, "upLast30days"], errors="coerce").fill
        dns = pd.to_numeric(revis.loc[rp, "downLast30days"], errors="coerce").fill
        y = np.arange(len(rp))
        axR.barh(y + 0.18, ups, height=0.35, color=POSITIVE, label="Up, 30d")
        axR.barh(y - 0.18, -dns, height=0.35, color=NEGATIVE, label="Down, 30d")
        axR.set_yticks(y); axR.set_yticklabels(rp)
        axR.axvline(0, color="#555", lw=0.9)
        axR.set_title("Analyst revisions, last 30 days")
        axR.set_xlabel("Number of analysts")
        axR.legend()
        axR.xaxis.set_major_formatter(mticker.FuncFormatter(lambda v, p: f"{abs(v):
plt.tight_layout()
plt.show()
```



```
In [17]: est_bits = []

if trend_t is not None and "0q" in trend_t.index:
    cur_e = pd.to_numeric(trend_t.loc["0q", "current"], errors="coerce")
    old_e = pd.to_numeric(trend_t.loc["0q", "90daysAgo"], errors="coerce")
    if pd.notna(cur_e) and pd.notna(old_e) and old_e != 0:
        drift = (cur_e / old_e - 1) * 100
        way = "raised" if drift > 0 else "cut"
        est_bits.append(
            f"- Current-quarter consensus EPS sits at **${cur_e:,.2f}**, "
            f"**{way}** {abs(drift):.1f}% from ${old_e:,.2f} ninety days ago."
        )

if revis is not None and "0q" in revis.index:
    u = pd.to_numeric(revis.loc["0q", "upLast30days"], errors="coerce")
    d = pd.to_numeric(revis.loc["0q", "downLast30days"], errors="coerce")
    if pd.notna(u) and pd.notna(d):
        net = u - d
        tone = "net upgrades" if net > 0 else "net downgrades" if net < 0 else "no"
        est_bits.append(
            f"- Revision breadth over 30 days: **{u:,.0f} up, {d:,.0f} down** "
            f"({tone}). Breadth confirms or undercuts the drift above."
        )

if est is not None and "0q" in est.index:
    n = pd.to_numeric(est.loc["0q", "numberOfAnalysts"], errors="coerce")
    lo = pd.to_numeric(est.loc["0q", "low"], errors="coerce")
    hi = pd.to_numeric(est.loc["0q", "high"], errors="coerce")
    av = pd.to_numeric(est.loc["0q", "avg"], errors="coerce")
    if pd.notna(lo) and pd.notna(hi) and pd.notna(av) and av != 0:
        band = (hi - lo) / abs(av) * 100
        verdict = ("genuine disagreement about the cycle" if n >= 15
                    else "too thin a sample to read as disagreement")
        est_bits.append(
            f"- The current-quarter band runs ${lo:,.2f} to ${hi:,.2f}, "
            f"**{band:,.0f}% of the mean**, across **{n:,.0f} analysts** - {verdict}"
        )

if targets.get("current") and targets.get("mean"):
    cur, mean_t = targets["current"], targets["mean"]
    est_bits.append(
```

```

f"- Consensus target **${mean_t:,.2f}** against a last price of "
f"**${cur:,.2f}**, or **{(mean_t / cur - 1) * 100:+,.1f}%** implied, "
f"across a ${targets.get('low', float('nan')):,.2f}-"
f"${targets.get('high', float('nan')):,.2f} range."
)

note(
f"""
**Forward expectations – findings**

{chr(10).join(est_bits) if est_bits else "- Estimate tables returned empty from the

**Caveat:** every number above is an opinion, not an observation. Consensus is
the hurdle the stock is priced against, and the useful reading is the gap between
where it sits now and where it sat a quarter ago – not whether the level looks
high or low in isolation.
"""
)

```

Forward expectations — findings

- Current-quarter consensus EPS sits at **\$31.33, raised** 40.7% from \$22.26 ninety days ago.
- Revision breadth over 30 days: **26 up, 0 down** (net upgrades). Breadth confirms or undercuts the drift above.
- The current-quarter band runs 28.04to37.44, **30% of the mean**, across **31 analysts** — genuine disagreement about the cycle.
- Consensus target **\$1,507.79** against a last price of **\$892.67**, or **+68.9%** implied, across a 361.00—2,200.00 range.

Caveat: every number above is an opinion, not an observation. Consensus is the hurdle the stock is priced against, and the useful reading is the gap between where it sits now and where it sat a quarter ago — not whether the level looks high or low in isolation.

3.4 Options positioning

Options data is a **separate endpoint** from the estimate tables above: `mu.options` lists expiration dates, `mu.option_chain(date)` returns `.calls`, `.puts` and `.underlying` for one expiry.

Only live chains exist. Once an expiration settles, its chain disappears from Yahoo and there is no historical archive in `yfinance` at any price. Questions of the form "what did positioning look like three months ago" need a paid vendor (ORATS, CBOE DataShop) or a snapshot job you run daily. This section covers live expirations only, which is forward-looking positioning.

Why the nearest expirations are often unusable. Open interest is not a live figure. The OCC computes it from settled positions overnight, and Yahoo serves the previous session's number. Contracts listed very recently — the daily and weekly series that dominate the front

of the expiration list for an active name — can therefore report **zero open interest even while trading actively**, because no settlement cycle has yet assigned them one. Taking `expirations[:3]` blindly walks straight into this.

The cell below therefore **probes several expirations, measures which ones actually carry open interest, and selects on that basis** rather than on proximity. If no expiry carries usable open interest, it falls back to volume and labels the charts accordingly instead of drawing empty bars.

What each measure means.

- **Open interest** — contracts currently held open, a stock of positions accumulated over the contract's life. The positioning measure.
- **Volume** — contracts traded in the session, a flow measure. Noisier, but live rather than lagged by a settlement cycle.

Put/call ratio. Above 1.0 means more puts than calls. The instinctive read is "bearish" and it is frequently wrong: heavy put open interest is equally consistent with holders buying protection on a position they intend to keep. Treat it as hedging demand, not directional opinion.

```
In [18]: all_exp = list(safe(lambda: mu.options, "options expirations", ()) or ())
PROBE = min(len(all_exp), 8)
print(f"{len(all_exp)} expirations listed; probing the first {PROBE}")

frames = []
for e in all_exp[:PROBE]:
    ch = safe(lambda d=e: mu.option_chain(d), f"chain {e}")
    if ch is None:
        continue
    for side, df in (("call", ch.calls), ("put", ch.puts)):
        d = df.copy()
        d["side"], d["expiry"] = side, e
        frames.append(d)

probe = pd.concat(frames, ignore_index=True) if frames else pd.DataFrame()

# ---- Diagnostic: distinguish "column missing" from "all null" from "true zero"
if not probe.empty:
    for c in ("openInterest", "volume"):
        if c not in probe.columns:
            print(f"[warn] column '{c}' absent from the chain")
            probe[c] = np.nan
            probe[c] = pd.to_numeric(probe[c], errors="coerce")

    diag = probe.groupby("expiry").agg(
        contracts=("strike", "size"),
        oi_null=("openInterest", lambda s: int(s.isna().sum())),
        oi_zero=("openInterest", lambda s: int((s == 0).sum())),
        oi_total=("openInterest", "sum"),
        vol_total=("volume", "sum"),
```

```

)
diag["oi_usable_pct"] = (
    (diag["contracts"] - diag["oi_null"] - diag["oi_zero"]) / diag["contracts"]
).round(1)
print("\nData quality by expiration")
display(diag)

```

22 expirations listed; probing the first 8

Data quality by expiration

	contracts	oi_null	oi_zero	oi_total	vol_total	oi_usable_pct
expiry						
2026-08-05	310	0	310	0	281879.0	0.0
2026-08-07	658	0	658	0	174733.0	0.0
2026-08-10	257	0	257	0	19065.0	0.0
2026-08-12	227	0	227	0	4395.0	0.0
2026-08-14	542	0	542	0	45016.0	0.0
2026-08-21	544	0	532	313	41891.0	2.2
2026-08-28	453	0	453	0	10915.0	0.0
2026-09-04	354	0	354	0	6455.0	0.0

```

In [19]: # ---- Select expirations that actually carry data -----
MIN_OI = 100          # an expiry below this is treated as not yet populated
N_EXPIRIES = 3

if probe.empty:
    chain, chosen, METRIC = pd.DataFrame(), [], "volume"
else:
    usable = diag[diag["oi_total"] >= MIN_OI]
    if len(usable) >= 1:
        METRIC = "openInterest"
        # Nearest expirations that clear the bar, not the largest - proximity
        # still matters, we are only excluding the unpopulated ones.
        chosen = list(usable.index[:N_EXPIRIES])
    else:
        METRIC = "volume"
        chosen = list(diag.sort_values("vol_total", ascending=False)
                      .index[:N_EXPIRIES].sort_values())
    print("[warn] no expiration carries usable open interest; "
          "falling back to volume. This is normal shortly after new weekly "
          "contracts list, before the first settlement cycle assigns OI.")

    chain = probe[probe.expiry.isin(chosen)].copy()
    chain[["openInterest", "volume"]] = chain[["openInterest", "volume"]].fillna(0)

METRIC_LABEL = ("Open interest (positions held)" if METRIC == "openInterest"
               else "Volume (contracts traded)")

```



```

if not chain.empty:
    print(f"\nUsing {chosen} | primary measure: {METRIC}")
    print(f"{len(chain):,} contracts across {chain.expiry.nunique()} expirations")
    display(chain.groupby(["expiry", "side"])[["openInterest", "volume"]]
              .sum().astype(int))

```

Using ['2026-08-21'] | primary measure: openInterest
 544 contracts across 1 expirations

		openInterest	volume
expiry side			
2026-08-21	call	226	24929
	put	87	16962

```

In [20]: if not chain.empty:
    agg = (chain.groupby(["expiry", "side"])[["openInterest", "volume"]]
           .sum().unstack("side").fillna(0))

    # Only plot a panel if it has data; an all-zero panel is worse than no panel.
    panels = [(m, lab) for m, lab in
               [("openInterest", "Open interest (positions held)"),
                ("volume", "Volume (contracts traded)")]
               if agg[m].to_numpy().sum() > 0]

    if not panels:
        print("[warn] neither measure has data for the selected expirations")
    else:
        fig, axes = plt.subplots(1, len(panels), figsize=(7 * len(panels), 5.5),
                                squeeze=False)
        x = np.arange(len(agg.index))
        w = 0.38

        for ax, (metric, title) in zip(axes[0], panels):
            calls = agg[(metric, "call")] if (metric, "call") in agg else pd.Series
            puts = agg[(metric, "put")] if (metric, "put") in agg else pd.Series
            b1 = ax.bar(x - w/2, calls.values, w, color=POSITIVE, label="Calls")
            b2 = ax.bar(x + w/2, puts.values, w, color=NEGATIVE, label="Puts")
            for bars in (b1, b2):
                for b in bars:
                    if b.get_height() > 0:
                        ax.text(b.get_x() + b.get_width()/2, b.get_height(),
                                f"{b.get_height():.0f}", ha="center", va="bottom",
                                fontsize=8)

            ax.set_xticks(x)
            ax.set_xticklabels([str(i) for i in agg.index])
            ax.set_title(title)
            ax.set_ylabel("Contracts")
            ax.legend()
            ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v, p: f"{v:,.0f}"))

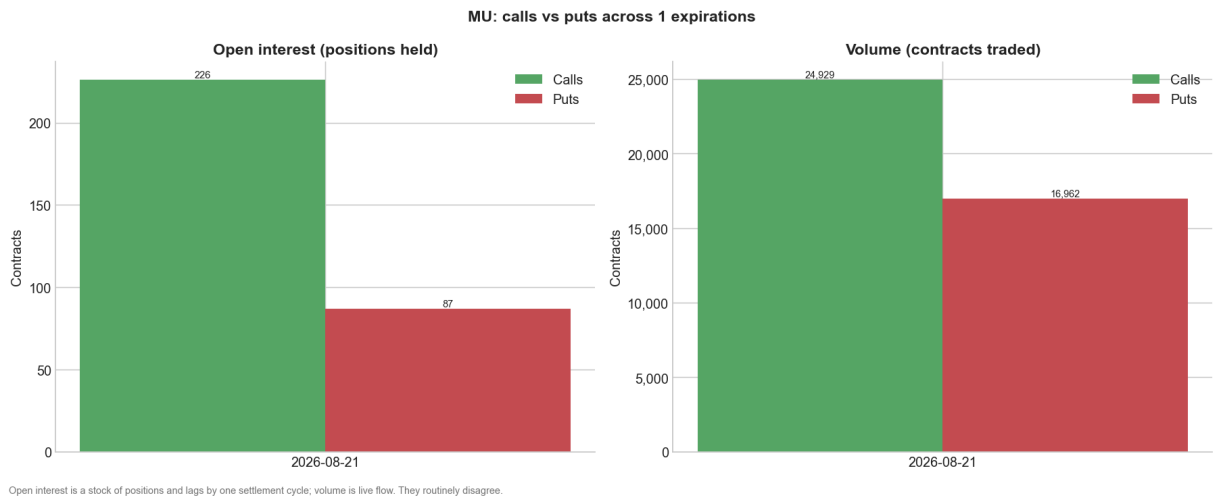
        fig.suptitle(f"{FOCUS_TICKER}: calls vs puts across {len(agg)} expirations"
                    fontsize=13, fontweight="bold")
        caption = ("Open interest is a stock of positions and lags by one settlement
                    cycle; volume is live flow. They routinely disagree.")

```

```

if METRIC == "volume":
    caption = ("Open interest was unpopulated for these expirations, so vol
               "is the primary measure here. " + caption)
fig.text(0.01, -0.02, caption, fontsize=8, color="#777")
plt.tight_layout()
plt.show()

```



3.4.1 Where positioning sits relative to spot

Aggregate counts say how much is on, not where. Plotting the primary measure by strike shows the levels traders actually chose, which is the more informative view — clusters sit at round numbers and at whatever was at-the-money when the positions were opened.

```

In [22]: if not chain.empty:
    near = chain[chain.expiry == chosen[0]].copy()

    spot = np.nan
    for getter in (lambda: float(mu.fast_info["last_price"]),
                   lambda: float(tech["close"].iloc[-1]),
                   lambda: float(near["lastPrice"].median())):
        try:
            spot = getter()
            if np.isfinite(spot) and spot > 0:
                break
        except Exception:
            continue

    near = near[(near.strike >= spot * 0.65) & (near.strike <= spot * 1.45)]

    if near[METRIC].sum() == 0:
        print(f"[warn] {METRIC} is empty for {chosen[0]}; skipping strike chart")
    else:
        # Strike spacing varies by name and expiry; derive bar width from the data
        # rather than hardcoding it, or bars overlap or vanish.
        strikes = np.sort(near.strike.unique())
        step = np.median(np.diff(strikes)) if len(strikes) > 1 else 1.0

        fig, ax = plt.subplots(figsize=(13, 5.5))
        for side, colour, sign in (("call", POSITIVE, 1), ("put", NEGATIVE, -1)):

```

```

d = near[near.side == side].groupby("strike")[METRIC].sum()
ax.bar(d.index, sign * d.values, width=step * 0.85, color=colour,
       label=f"{side.capitalize()}s")

ax.axvline(spot, color="#1f1f1f", ls="--", lw=1.3)
ax.annotate(f"spot ${spot:,.2f}", (spot, ax.get_ylim()[1] * 0.85),
           xytext=(6, 0), textcoords="offset points", fontsize=9)
ax.axhline(0, color="#555", lw=0.9)
ax.set_xlabel("Strike ($)")
ax.set_ylabel(f"{METRIC_LABEL} (calls up, puts down)")
ax.set_title(f"{METRIC_LABEL} by strike, {chosen[0]} expiry")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v, p: f"{abs(v):,}"))
ax.legend()
plt.tight_layout()
plt.show()

```

[warn] openInterest is empty for 2026-08-21; skipping strike chart

```

In [23]: if not chain.empty:
    tot = chain.groupby("side")[METRIC].sum()
    calls_t, puts_t = float(tot.get("call", 0)), float(tot.get("put", 0))
    pc = puts_t / calls_t if calls_t else float("nan")

    by_exp = chain.groupby(["expiry", "side"])[METRIC].sum().unstack("side")
    by_exp["put_call"] = (by_exp.get("put") / by_exp.get("call")).replace(0, np.nan)

    lean = ("hedging demand or bearish positioning" if pc > 1.2
            else "upside positioning" if pc < 0.8
            else "no meaningful skew")
    measure_note = (" " if METRIC == "openInterest" else
                    " Note that this reads volume, not open interest, because open
                    interest was unpopulated for these expirations – it reflects o
                    session of flow rather than standing positions.")

    note(
f"""
**Options positioning – findings**

- Across {' '.join(chosen)}, total {METRIC_LABEL.split(' ')[0].lower()} is
  **{calls_t:,.0f} calls** against **{puts_t:,.0f} puts** – a put/call ratio of
  **{pc:,.2f}**, pointing to {lean}.{measure_note}
- Per-expiry put/call:
  {' '.join(f"{i} = {v:,.2f}" for i, v in by_exp['put_call'].dropna().items())}.

**Caveat:** a high put/call ratio is not automatically bearish. Protective puts
held against a long position produce the same reading as outright bearish bets,
and this feed cannot distinguish the two.
"""
    )

```

Options positioning — findings

- Across 2026-08-21, total open interest is **226 calls** against **87 puts** — a put/call ratio of **0.38**, pointing to upside positioning.
- Per-expiry put/call: 2026-08-21 = 0.38.

Caveat: a high put/call ratio is not automatically bearish. Protective puts held against a long position produce the same reading as outright bearish bets, and this feed cannot distinguish the two.

3.5 News sentiment

Two engines are run over the same headlines and compared, because they fail in different ways and the disagreement is itself the interesting output.

- **VADER** (`nltk`) — a rule-based lexicon built for social media. Instant, no model download. It does not know finance: "downgrade", "guidance cut" and "beats consensus" carry little or no weight in its lexicon, so it leans on generic intensity words and punctuation.
- **FinBERT** (`ProsusAI/finbert`) — BERT fine-tuned on financial text. It has learned that "misses estimates" is negative and "raises outlook" is positive. Roughly a 400MB download on first run, then a second or two per batch.

Read this section as qualitative, not as a metric. `mu.news` returns about ten items, titles only, over an uncontrolled window — often not a clean week. A mean sentiment score over $n=10$ headlines has no statistical meaning and will swing on a single article. It is presentable as a colour-coded table of what is being said. It is not presentable as a number to track over time. A real sentiment series needs a wider feed (NewsAPI, GDELT, Finnhub) and a rolling window measured in hundreds of items.

Install once:

```
pip install nltk transformers torch
```

```
In [24]: raw_news = safe(lambda: mu.news, "news", []) or []

# yfinance nests articles under 'content'; flat-key code raises KeyError here.
rows = []
for n in raw_news:
    c = n.get("content", n)
    rows.append({
        "date": (c.get("pubDate") or "")[:10],
        "provider": (c.get("provider") or {}).get("displayName", ""),
        "title": c.get("title", ""),
    })

news = pd.DataFrame(rows)
if not news.empty:
```

```

news["date"] = pd.to_datetime(news["date"], errors="coerce")
news = news.dropna(subset=["title"]).sort_values("date", ascending=False)
cutoff = pd.Timestamp.today().normalize() - pd.Timedelta(days=7)
news["this_week"] = news["date"] >= cutoff
print(f"{len(news)} headlines | {int(news.this_week.sum())} within the last 7 d
display(news.head(12).reset_index(drop=True))

```

10 headlines | 10 within the last 7 days

	date	provider	title	this_week
0	2026-08-05	Stocktwits	MU, SNDK, SKHY Stocks In Focus: Elon Musk Says...	True
1	2026-08-05	Motley Fool	Is SK Hynix Quietly Winning the AI Memory Supe...	True
2	2026-08-05	Motley Fool	Is Micron Stock Going Higher? You Need To Hear...	True
3	2026-08-05	Bloomberg	SK Hynix Climbs as Investors Bet on Shareholde...	True
4	2026-08-05	Motley Fool	Micron Trades at Just 5 Times Forward Earnings...	True
5	2026-08-04	Yahoo Finance	Sandisk bulls are probably eyeing this number ...	True
6	2026-08-04	Yahoo Finance	Marvell, Sandisk, SK Hynix lead semiconductor ...	True
7	2026-08-04	Zacks	Top Stock Reports for Berkshire Hathaway, Micr...	True
8	2026-08-04	Barrons.com	SpaceX, Palantir, AMD, Marvell, Caterpillar, P...	True
9	2026-08-04	Barrons.com	Micron Stock Rises as It Gains Memory-Chip Sha...	True

```

In [26]: # ---- VADER -----
vader = None
try:
    import nltk
    from nltk.sentiment.vader import SentimentIntensityAnalyzer
    try:
        vader = SentimentIntensityAnalyzer()
    except LookupError:
        nltk.download("vader_lexicon", quiet=True)
        vader = SentimentIntensityAnalyzer()
except Exception as exc:
    print(f"[warn] VADER unavailable: {exc}")

# ---- FinBERT -----
finbert = None
try:
    from transformers import pipeline
    finbert = pipeline("text-classification", model="ProsusAI/finbert",
                       truncation=True, top_k=None)
except Exception as exc:
    print(f"[warn] FinBERT unavailable: {exc}")

def finbert_score(text):
    # Signed score in [-1, 1]: P(positive) - P(negative). Neutral mass pulls to 0.
    out = finbert(text)[0]
    d = {r["label"].lower(): r["score"] for r in out}
    return d.get("positive", 0) - d.get("negative", 0)

```

```

if not news.empty:
    if vader is not None:
        news["vader"] = news["title"].map(lambda t: vader.polarity_scores(t)["compo
    if finbert is not None:
        news["finbert"] = news["title"].map(finbert_score)

    def bucket(x, lo=-0.15, hi=0.15):
        return "positive" if x > hi else "negative" if x < lo else "neutral"

    for col in ("vader", "finbert"):
        if col in news:
            news[f"{col}_label"] = news[col].map(bucket)

display(news[[c for c in ["date", "provider", "title", "vader", "finbert",
                        "vader_label", "finbert_label"] if c in news.columns]]
        .head(12).round(3).reset_index(drop=True))

```

Device set to use cuda:0

	date	provider	title	vader	finbert	vader_label	finbert_label
0	2026-08-05	Stocktwits	MU, SNDK, SKHY Stocks In Focus: Elon Musk Says...	0.052	0.841	neutral	positive
1	2026-08-05	Motley Fool	Is SK Hynix Quietly Winning the AI Memory Supe...	0.527	0.293	positive	positive
2	2026-08-05	Motley Fool	Is Micron Stock Going Higher? You Need To Hear...	0.000	0.060	neutral	neutral
3	2026-08-05	Bloomberg	SK Hynix Climbs as Investors Bet on Shareholde...	0.000	0.844	neutral	positive
4	2026-08-05	Motley Fool	Micron Trades at Just 5 Times Forward Earnings...	-0.402	-0.009	negative	neutral
5	2026-08-04	Yahoo Finance	Sandisk bulls are probably eyeing this number ...	0.096	0.095	neutral	neutral
6	2026-08-04	Yahoo Finance	Marvell, Sandisk, SK Hynix lead semiconductor ...	0.000	0.578	neutral	positive
7	2026-08-04	Zacks	Top Stock Reports for Berkshire Hathaway, Micr...	0.202	-0.020	positive	neutral
8	2026-08-04	Barrons.com	SpaceX, Palantir, AMD, Marvell, Caterpillar, P...	0.000	0.013	neutral	neutral
9	2026-08-04	Barrons.com	Micron Stock Rises as It Gains Memory-Chip Sha...	0.295	0.430	positive	positive

```
In [27]: have_both = {"vader", "finbert"}.issubset(news.columns) if not news.empty else False

if have_both:
    fig, (axA, axB) = plt.subplots(1, 2, figsize=(14, 5.5))

    y = np.arange(len(news))
    axA.barh(y - 0.2, news["vader"], height=0.38, color=MUTED, label="VADER")
    axA.barh(y + 0.2, news["finbert"], height=0.38, color=ACCENT, label="FinBERT")
    axA.axvline(0, color="#555", lw=0.9)
    axA.set_yticks(y)
    axA.set_yticklabels([t[:44] + ("..." if len(t) > 44 else "")
                        for t in news["title"]], fontsize=7)

    axA.invert_yaxis()
    axA.set_xlim(-1, 1)
    axA.set_xlabel("Sentiment (-1 to +1)")
    axA.set_title("Headline by headline")
```

```

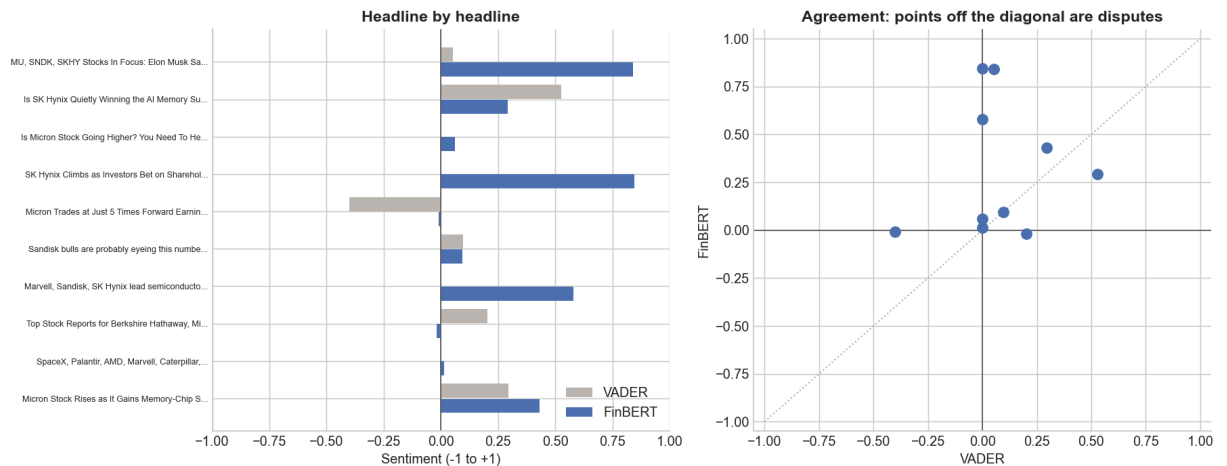
axA.legend(loc="lower right")

axB.scatter(news["vader"], news["finbert"], s=70, color=ACCENT, zorder=3)
axB.axhline(0, color="#555", lw=0.9)
axB.axvline(0, color="#555", lw=0.9)
axB.plot([-1, 1], [-1, 1], color=MUTED, ls=":", lw=1.2)
axB.set_xlim(-1.05, 1.05); axB.set_ylim(-1.05, 1.05)
axB.set_xlabel("VADER"); axB.set_ylabel("FinBERT")
axB.set_title("Agreement: points off the diagonal are disputes")
plt.tight_layout()
plt.show()

xt = pd.crosstab(news["vader_label"], news["finbert_label"])
print("VADER (rows) vs FinBERT (columns)")
display(xt)

disputes = news[news["vader_label"] != news["finbert_label"]]
if not disputes.empty:
    print(f"\n{len(disputes)} of {len(news)} headlines classified differently:")
    display(disputes[["title", "vader", "finbert", "vader_label", "finbert_label"]
                    .round(3).reset_index(drop=True)])

```



VADER (rows) vs FinBERT (columns)

finbert_label neutral positive

vader_label		
negative	1	0
neutral	3	3
positive	1	2

5 of 10 headlines classified differently:

	title	vader	finbert	vader_label	finbert_label
0	MU, SNDK, SKHY Stocks In Focus: Elon Musk Says...	0.052	0.841	neutral	positive
1	SK Hynix Climbs as Investors Bet on Shareholde...	0.000	0.844	neutral	positive
2	Micron Trades at Just 5 Times Forward Earnings...	-0.402	-0.009	negative	neutral
3	Marvell, Sandisk, SK Hynix lead semiconductor ...	0.000	0.578	neutral	positive
4	Top Stock Reports for Berkshire Hathaway, Micr...	0.202	-0.020	positive	neutral

```
In [28]: sent_bits = []
if not news.empty:
    n_week = int(news["this_week"].sum())
    sent_bits.append(
        f"- **{len(news)} headlines** returned, **{n_week}** of them inside the "
        f"last seven days. The feed does not honour a date filter, so the weekly "
        f"subset is whatever happened to be published, not a sample."
    )
    for col, name in (("finbert", "FinBERT"), ("vader", "VADER")):
        if col in news.columns:
            sub = news[news["this_week"]] if n_week >= 3 else news
            counts = sub[f"{col}_label"].value_counts()
            sent_bits.append(
                f"- **{name}** reads the set as {counts.get('positive', 0)} positiv
                f"{counts.get('neutral', 0)} neutral, {counts.get('negative', 0)} "
                f"negative (mean {sub[col].mean():+.3f})."
            )
    if have_both:
        agree = (news["vader_label"] == news["finbert_label"]).mean() * 100
        corr = news["vader"].corr(news["finbert"])
        sent_bits.append(
            f"- The two engines agree on **{agree:,.0f}%** of headlines "
            f"(correlation {corr:+.2f}). Where they split, FinBERT is usually "
            f"right: it recognises financial phrasing that VADER's general-purpose "
            f"lexicon scores as neutral."
        )

note(
    f"""
    **News sentiment – findings**

    {chr(10).join(sent_bits) if sent_bits else "- No headlines returned by the feed."}

    **Do not treat this as a metric.** With a sample this small, one article moves
    the mean materially, and the window is uncontrolled. The table above is a
    qualitative summary of what is being written, useful as context alongside the
    revision data in 3.3 – which is a genuine measurement – and not as a number to
    trend.
    """
)
```

News sentiment — findings

- **10 headlines** returned, **10** of them inside the last seven days. The feed does not honour a date filter, so the weekly subset is whatever happened to be published, not a sample.
- **FinBERT** reads the set as 5 positive, 5 neutral, 0 negative (mean +0.313).
- **VADER** reads the set as 3 positive, 6 neutral, 1 negative (mean +0.077).
- The two engines agree on **50%** of headlines (correlation +0.12). Where they split, FinBERT is usually right: it recognises financial phrasing that VADER's general-purpose lexicon scores as neutral.

Do not treat this as a metric. With a sample this small, one article moves the mean materially, and the window is uncontrolled. The table above is a qualitative summary of what is being written, useful as context alongside the revision data in 3.3 — which is a genuine measurement — and not as a number to trend.

3.5.1 Analyst actions

Ratings changes are a slower, higher-conviction signal than headline sentiment: a firm changing its rating has committed a published model to the view.

Yahoo's `Action` field is unreliable — it is frequently null even when `Firm`, `FromGrade` and `ToGrade` are all populated, which produces an empty chart from a table that visibly has data. The cell below therefore **derives direction from the grade transition itself** by ranking both grades on a common scale, and falls back to the `Action` string only where a grade is unrecognised.

```
In [29]: ud = safe(lambda: mu.upgrades_downgrades, "upgrades / downgrades")

GRADE_RANK = {
    "strong sell": 0, "sell": 1, "underperform": 1, "underweight": 1,
    "reduce": 1, "negative": 1,
    "hold": 2, "neutral": 2, "equal-weight": 2, "equalweight": 2, "peer perform": 2,
    "market perform": 2, "sector perform": 2, "in-line": 2, "perform": 2,
    "buy": 3, "outperform": 3, "overweight": 3, "accumulate": 3, "add": 3,
    "positive": 3, "sector outperform": 3, "market outperform": 3,
    "strong buy": 4, "conviction buy": 4, "top pick": 4,
}

def grade_rank(g):
    return GRADE_RANK.get(str(g).strip().lower())

def classify(row):
    to_r, from_r = grade_rank(row.get("ToGrade")), grade_rank(row.get("FromGrade"))
    if to_r is not None and from_r is not None:
        if to_r > from_r: return "Upgrade"
        if to_r < from_r: return "Downgrade"
        return "Reiterate"
    action = str(row.get("Action", "")).strip().lower()
    return {"up": "Upgrade", "down": "Downgrade", "reit": "Reiterate",
```

```

        "main": "Reiterate", "init": "Initiate"}.get(action, "Unclassified")

if ud is not None and not ud.empty:
    recent = ud.sort_index(ascending=False).head(15).copy()
    recent["direction"] = recent.apply(classify, axis=1)

    show_cols = [c for c in ["Firm", "FromGrade", "ToGrade", "Action", "direction"]
                  if c in recent.columns]
    display(recent[show_cols])

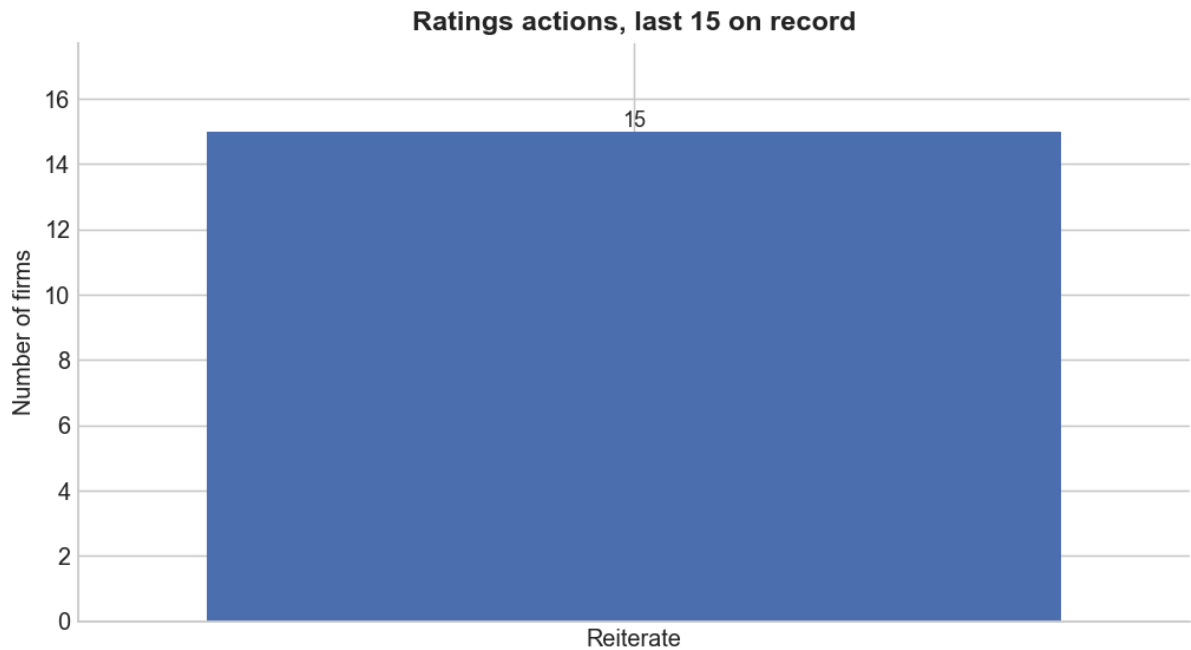
    order = ["Upgrade", "Reiterate", "Initiate", "Downgrade", "Unclassified"]
    counts = recent["direction"].value_counts().reindex(order).dropna()
    counts = counts[counts > 0]

    if counts.empty:
        print("[warn] no classifiable ratings actions")
    else:
        palette = {"Upgrade": POSITIVE, "Reiterate": ACCENT, "Initiate": MUTED,
                   "Downgrade": NEGATIVE, "Unclassified": "#D8D4CE"}
        fig, ax = plt.subplots(figsize=(8, 4.5))
        bars = ax.bar(counts.index, counts.values,
                      color=[palette[i] for i in counts.index])
        for b in bars:
            ax.text(b.get_x() + b.get_width()/2, b.get_height(),
                    f"{int(b.get_height())}", ha="center", va="bottom", fontsize=10)
        ax.set_title(f"Ratings actions, last {len(recent)} on record")
        ax.set_ylabel("Number of firms")
        ax.set_ylim(0, counts.max() * 1.18)
        ax.margins(x=0.15)
        fig.text(0.01, -0.03,
                "Direction derived from the FromGrade to ToGrade transition, not "
                "from Yahoo's Action field, which is often null.",
                fontsize=8, color="#777")
        plt.tight_layout()
        plt.show()

        n_up, n_dn = int(counts.get("Upgrade", 0)), int(counts.get("Downgrade", 0))
        note(
f"""
**Ratings actions – {n_up} upgrades against {n_dn} downgrades** across the last
{len(recent)} recorded changes. Ratings lag price and cluster after earnings, so
read this as confirmation of a move already made rather than as a leading signal.
"""
        )

```

GradeDate	Firm	FromGrade	ToGrade	Action	direction
2026-07-14 11:13:19	Keybanc	Overweight	Overweight	main	Reiterate
2026-06-29 14:43:21	Cantor Fitzgerald	Overweight	Overweight	main	Reiterate
2026-06-25 20:36:38	Barclays	Overweight	Overweight	main	Reiterate
2026-06-25 19:11:54	Citigroup	Buy	Buy	main	Reiterate
2026-06-25 17:51:26	Goldman Sachs	Neutral	Neutral	main	Reiterate
2026-06-25 17:29:15	Morgan Stanley	Overweight	Overweight	main	Reiterate
2026-06-25 17:05:30	Raymond James	Outperform	Outperform	main	Reiterate
2026-06-25 16:35:42	Deutsche Bank	Buy	Buy	main	Reiterate
2026-06-25 16:34:54	Wedbush	Outperform	Outperform	main	Reiterate
2026-06-25 16:19:26	Mizuho	Outperform	Outperform	main	Reiterate
2026-06-25 15:33:25	DA Davidson	Buy	Buy	main	Reiterate
2026-06-25 15:30:05	Wells Fargo	Overweight	Overweight	main	Reiterate
2026-06-25 15:13:04	RBC Capital	Outperform	Outperform	main	Reiterate
2026-06-25 14:39:40	TD Cowen	Buy	Buy	main	Reiterate
2026-06-25 14:01:40	Susquehanna	Positive	Positive	main	Reiterate



Direction derived from the FromGrade to ToGrade transition, not from Yahoo's Action field, which is often null.

Ratings actions — 0 upgrades against 0 downgrades across the last 15 recorded changes. Ratings lag price and cluster after earnings, so read this as confirmation of a move already made rather than as a leading signal.

3.6 Fundamental conclusion

```
In [30]: bits = []

if fin is not None and len(fin) >= 2:
    gm_now, gm_prev = fin["gross_margin_pct"].iloc[-1], fin["gross_margin_pct"].iloc[-2]
    gm_min, gm_max = fin["gross_margin_pct"].min(), fin["gross_margin_pct"].max()
    direction = "expanding" if gm_now > gm_prev else "contracting"
    bits.append(
        f"- Gross margin is **{gm_now:,.1f}%**, **{direction}** from "
        f"{gm_prev:,.1f}% in the prior quarter, against a range of {gm_min:,.1f}% "
        f"to {gm_max:,.1f}% across available history."
    )
    if len(fin) >= 5:
        rev_yoy = (fin["revenue"].iloc[-1] / fin["revenue"].iloc[-5] - 1) * 100
        bits.append(f"- Revenue is **{rev_yoy:+.1f}%** year over year.")

if info.get("trailingPE") and info.get("forwardPE"):
    tp, fp = info["trailingPE"], info["forwardPE"]
    bits.append(
        f"- Forward P/E of **{fp:,.1f}%** sits {'below' if fp < tp else 'above'} the "
        f"trailing **{tp:,.1f}%**, implying the market expects earnings to "
        f"{'rise' if fp < tp else 'fall'} over the next twelve months."
    )

if targets.get("current") and targets.get("mean"):
    cur, mean_t = targets["current"], targets["mean"]
    bits.append(
        f"- Consensus target is **${mean_t:,.2f}%** against a last price of "
        f"**${cur:,.2f}%**, or **{(mean_t / cur - 1) * 100:+.1f}%** implied. "
        f"The range runs ${targets.get('low', float('nan')):,.2f} to "
        f"${targets.get('high', float('nan')):,.2f} – a band that is itself a "
        f"measure of disagreement."
    )

body = "\n".join(bits) if bits else "- Insufficient fundamental data returned by th

note(
    f"""
    **{FOCUS_TICKER} fundamentals – findings**

    {body}

    **Caveat:** memory is a cyclical business. Trailing multiples look cheapest at the
    cycle peak, when trailing earnings sit at a maximum unlikely to repeat, and most
    expensive at the trough. Margin direction is a more reliable cycle indicator here
    than any P/E reading.
    """
)
```

MU fundamentals — findings

- Gross margin is **84.6%**, **expanding** from 74.4% in the prior quarter, against a range of 37.7% to 84.6% across available history.
- Revenue is **+345.7%** year over year.
- Forward P/E of **5.7** sits below the trailing **18.8**, implying the market expects earnings to rise over the next twelve months.
- Consensus target is **\$1,507.79** against a last price of **\$892.67**, or **+68.9%** implied. The range runs 361.00 to 2,200.00 — a band that is itself a measure of disagreement.

Caveat: memory is a cyclical business. Trailing multiples look cheapest at the cycle peak, when trailing earnings sit at a maximum unlikely to repeat, and most expensive at the trough. Margin direction is a more reliable cycle indicator here than any P/E reading.

4. Technicals — MU

Timing layer only. Everything below describes the price series; none of it speaks to the business. Two panels sharing a time axis: price with trend references, and RSI beneath.

```
In [31]: mu_full = get_close(FOCUS_TICKER, LOOKBACK_START)[FOCUS_TICKER]

tech = pd.DataFrame({"close": mu_full})
tech["ema8"] = mu_full.ewm(span=8, adjust=False).mean()
tech["ema21"] = mu_full.ewm(span=21, adjust=False).mean()
tech["sma50"] = mu_full.rolling(50).mean()
tech["sma200"] = mu_full.rolling(200).mean()
tech["rsi14"] = rsi(mu_full)

# Computed on full history, displayed from DISPLAY_START, so there is no warm-up gap
tech = tech.loc[DISPLAY_START:]

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(13, 9), sharex=True,
                              gridspec_kw={"height_ratios": [3, 1]})

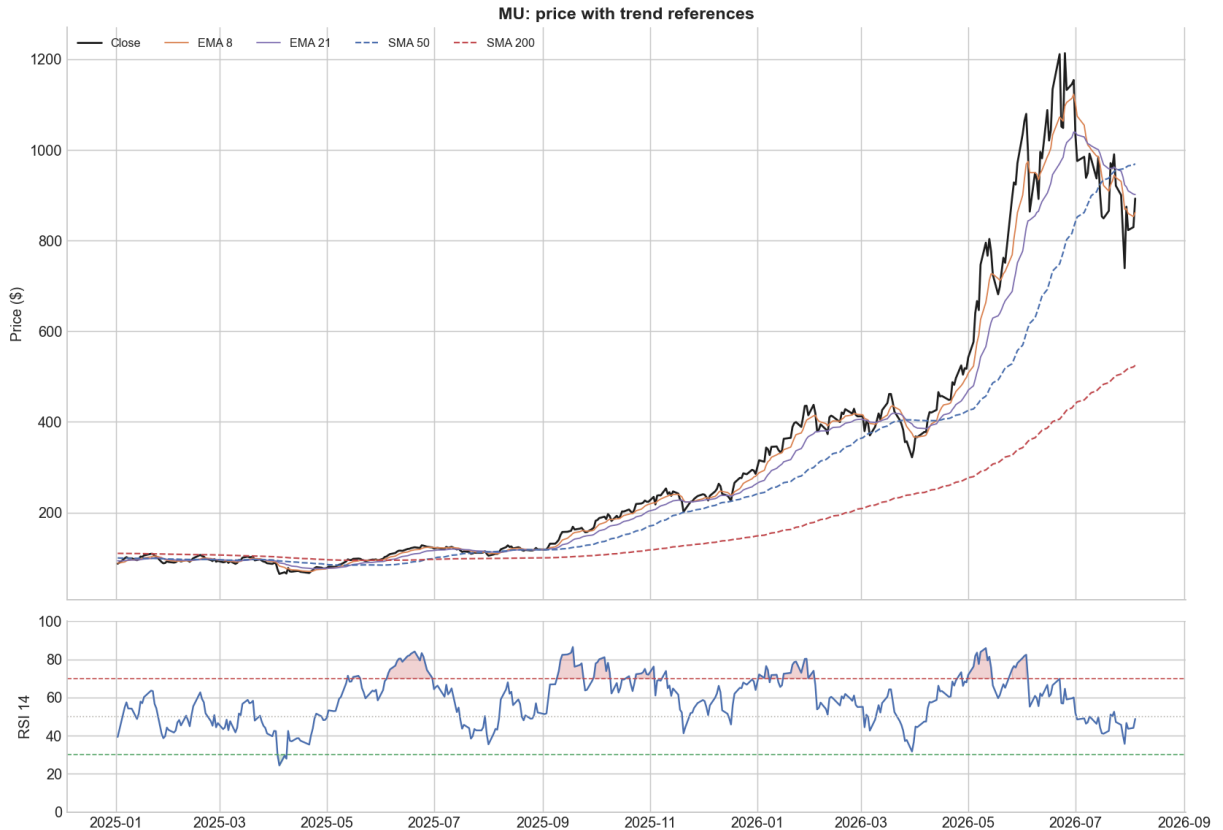
ax1.plot(tech.index, tech["close"], color="#1f1f1f", lw=1.5, label="Close")
ax1.plot(tech.index, tech["ema8"], color="#DD8452", lw=1.0, label="EMA 8")
ax1.plot(tech.index, tech["ema21"], color="#8172B3", lw=1.0, label="EMA 21")
ax1.plot(tech.index, tech["sma50"], color=ACCENT, lw=1.2, ls="--", label="SMA 50")
ax1.plot(tech.index, tech["sma200"], color=NEGATIVE, lw=1.2, ls="--", label="SMA 200")
ax1.set_ylabel("Price ($)")
ax1.set_title(f"{FOCUS_TICKER}: price with trend references")
ax1.legend(loc="upper left", ncol=5, fontsize=9)

ax2.plot(tech.index, tech["rsi14"], color=ACCENT, lw=1.3)
ax2.axhline(70, color=NEGATIVE, ls="--", lw=0.9)
ax2.axhline(30, color=POSITIVE, ls="--", lw=0.9)
ax2.axhline(50, color=MUTED, ls=":", lw=0.9)
ax2.fill_between(tech.index, 70, tech["rsi14"], where=tech["rsi14"] >= 70,
```

```

        color=NEGATIVE, alpha=0.25)
ax2.fill_between(tech.index, 30, tech["rsi14"], where=tech["rsi14"] <= 30,
        color=POSITIVE, alpha=0.25)
ax2.set_ylim(0, 100)
ax2.set_ylabel("RSI 14")
ax2.set_xlabel("")
plt.tight_layout()
plt.show()

```



4.1 Technical conclusion

```

In [32]: last      = tech.iloc[-1]
above_200 = last["close"] > last["sma200"]
above_50  = last["close"] > last["sma50"]
fast_cross = last["ema8"] > last["ema21"]
dist_200   = (last["close"] / last["sma200"] - 1) * 100
r          = last["rsi14"]
regime     = ("overbought" if r >= 70 else "oversold" if r <= 30
              else "bullish-neutral" if r >= 50 else "bearish-neutral")
days_over_70 = int((tech["rsi14"] >= 70).sum())
stacked      = above_200 and above_50 and fast_cross

long_trend = ("above, the conventional definition of a long-term uptrend"
              if above_200 else
              "below, the conventional definition of a long-term downtrend")
structure  = ("All trend references are stacked bullishly."
              if stacked else
              "The trend references disagree, which usually means consolidation "
              "rather than a decisive move.")

```

```

note(
f"""
**{FOCUS_TICKER} technicals – findings**

- Last close **${last['close']:.2f}**, sitting **{dist_200:+.1f}%** relative to
  its 200-day average – {long_trend}.
- Short-term structure: price is **{'above' if above_50 else 'below'}** the
  50-day, and the 8-day EMA is **{'above' if fast_cross else 'below'}** the
  21-day. {structure}
- RSI-14 reads **{r:.1f}**, a **{regime}** regime. The indicator spent
  **{days_over_70} of {len(tech)}** displayed sessions above 70 – a reminder that
  in a strong trend an overbought reading persists rather than reverses.

**So what:** technicals inform entry timing only. They neither validate nor
contradict the fundamental picture in Section 3, and should not be read as doing so
"""
)

```

MU technicals — findings

- Last close **\$892.67**, sitting **+70.1%** relative to its 200-day average — above, the conventional definition of a long-term uptrend.
- Short-term structure: price is **below** the 50-day, and the 8-day EMA is **below** the 21-day. The trend references disagree, which usually means consolidation rather than a decisive move.
- RSI-14 reads **48.8**, a **bearish-neutral** regime. The indicator spent **85 of 397** displayed sessions above 70 — a reminder that in a strong trend an overbought reading persists rather than reverses.

So what: technicals inform entry timing only. They neither validate nor contradict the fundamental picture in Section 3, and should not be read as doing so.

5. Executive Summary

Placed last so it is generated from everything computed above, but meant to be read first. When exporting, move this section to the top of the PDF if the audience wants the conclusion before the evidence.

```

In [33]: margin_line = ""
if fin is not None and len(fin) >= 2:
    gm_last = fin["gross_margin_pct"].iloc[-1]
    gm_dir = "expanding" if gm_last > fin["gross_margin_pct"].iloc[-2] else "contr
    margin_line = f"Gross margin sits at {gm_last:.1f}% and is {gm_dir}."

note(
f"""
### Executive summary – {pd.Timestamp.today().date()}

```



```

**Window:** {DISPLAY_START} to {px.index.max().date()} ({len(px)} trading days)

1. **Market.** Sector dispersion ran **{spread:,.0f} points** between
   {leader} ({pct(perf.loc[leader, 'total_return_pct'])}) and
   {laggard} ({pct(perf.loc[laggard, 'total_return_pct'])}), with
   {above} of {len(perf)} sectors finishing positive. Selection mattered more
   than direction.

2. **Sector.** Technology's relative strength against the equal-weight basket of
   other sectors ended at **{rs_now:,.1f}**, having led for
   **{share_leading:,.0f}%** of sessions. The lead is currently **{trend}**.

3. **Security.** {FOCUS_TICKER} closed at **${last['close']:,.2f}**,
   **{dist_200:+,.1f}%** versus its 200-day average, RSI **{r:,.1f}**
   ({regime}). {margin_line}

**What this report does not do.** No forecast, no valuation model, no position
sizing, no recommendation. It is a descriptive snapshot of one window on a free
data feed — a different start date would produce different rankings — and every
sector conclusion rests on a single ETF standing in for an entire sector.
"""
)

```

Executive summary — 2026-08-05

Window: 2025-01-01 to 2026-08-04 (397 trading days)

1. **Market.** Sector dispersion ran **54 points** between XLK (62.5%) and XLY (8.2%), with 11 of 11 sectors finishing positive. Selection mattered more than direction.
2. **Sector.** Technology's relative strength against the equal-weight basket of other sectors ended at **131.5**, having led for **72%** of sessions. The lead is currently **still expanding**.
3. **Security.** MU closed at **\$892.67**, **+70.1%** versus its 200-day average, RSI **48.8** (bearish-neutral). Gross margin sits at 84.6% and is expanding.

What this report does not do. No forecast, no valuation model, no position sizing, no recommendation. It is a descriptive snapshot of one window on a free data feed — a different start date would produce different rankings — and every sector conclusion rests on a single ETF standing in for an entire sector.

In []: